

Bundle-to-Product Retrieval in Fashion: Technical Paper and System Documentation

Hack2026 Team
Internal Technical Report

Abstract—This document describes the full Hack2026 retrieval system for fashion bundle decomposition, where each bundle image must be matched to one or more product identifiers. The implementation combines a baseline pretrained visual retrieval pipeline and an advanced OpenCLIP training pipeline with optional clothing-region detection. We detail data preprocessing, offline augmentation, model training, local validation, inference, and submission generation. The paper is grounded on the repository codebase, with an emphasis on reproducibility and engineering decisions.

Index Terms—visual retrieval, multimodal learning, OpenCLIP, fashion AI, metric learning, object-level matching

I. INTRODUCTION

The Hack2026 task is a multi-label visual retrieval problem: given a bundle image (an outfit or set of garments), the system must predict the list of product identifiers (`product_asset_id`) present in that bundle. The final output is a CSV with one row per predicted product, grouped by `bundle_asset_id`, with a maximum of 15 predictions per bundle.

This repository implements two complementary approaches:

- 1) A lightweight pretrained baseline (`src/infer.py`) based on cosine retrieval over image embeddings from torchvision backbones.
- 2) A trainable multimodal retrieval model (`src/models/retrieval_openclip.py`) based on Marqo Fashion SigLIP, with optional bundle-region detection and contrastive optimization.

From an engineering perspective, the codebase is organized around three stages:

- 1) Data curation and asset preparation (`preprocess_data.py`, `download.sh`, `offline_augment.py`).
- 2) Training and local validation (`src/train.py` with Hydra config).
- 3) Test-time inference and submission export (`src/infer.py`, `src/infer_openclip.py`).

This paper documents the end-to-end pipeline and the practical decisions embedded in the current implementation.

II. DATA AND PREPROCESSING

A. Source Datasets

The core inputs are four CSV files. Their functional role in the project is summarized in Table I.

File	Purpose
<code>data/bundles_dataset.csv</code>	Bundle catalog and bundle image URLs.
<code>data/product_dataset.csv</code>	Product catalog, product image URLs, and textual descriptions.
<code>data/bundles_product_map.csv</code>	Supervised bundle-product links for training.
<code>data/bundles_product_map_test.csv</code>	Test set, where <code>product_asset_id</code> must be predicted.

TABLE I

REPOSITORY DATASETS USED BY TRAINING AND INFERENCE.

B. Preprocessing Pipeline

Script `preprocess_data.py` provides:

- 1) Schema checks for required columns and referential integrity.
- 2) Cleaning of product description labels, including exclusion of predefined cosmetic/perfume categories.
- 3) Stratified train/validation split by number of labels per bundle.
- 4) Concurrent image download with retry logic and forbidden URL logging.
- 5) JSONL manifest generation and preprocessing statistics.

The script produces:

- `manifests/train_manifest.jsonl`
- `manifests/val_manifest.jsonl`
- `labels.json`, `label2idx.json`, and `stats.json`

C. Offline Augmentation

Script `offline_augment.py` generates deterministic, seed-controlled augmented views for both bundles and products. It applies resized crops, color jitter, optional horizontal flips, Gaussian blur, optional cutout (bundles), and JPEG quality perturbation. Augmented images and manifests are saved as:

- `bundles_aug/*.jpg`, `bundles_aug_manifest.jsonl`
- `products_aug/*.jpg`, `products_aug_manifest.jsonl`

This design is aligned with the one-image-per-product constraint, where robust product representations require synthetic view expansion.

III. METHODOLOGY

A. Problem Formulation

For each bundle query b , the objective is to rank product candidates $\{p_i\}_{i=1}^N$ and output up to 15 unique product IDs with highest relevance. Training uses known positive links from `bundles_product_match_train.csv`.

B. Baseline Retrieval (`src/infer.py`)

The baseline uses pretrained torchvision backbones (`resnet18`, `resnet50`, `efficientnet_b0`) to encode bundle and product images. Embeddings are L2-normalized and cosine similarity is computed by dot product.

For each query embedding $\mathbf{q}$ and product matrix $\mathbf{P}$:

$$\text{sim}(\mathbf{q}, \mathbf{P}) = \mathbf{q}\mathbf{P}^\top \quad (1)$$

Top- K retrieval is implemented via partial sorting (`np.argsort`) for speed.

C. OpenCLIP Multimodal Retrieval

The advanced backend uses Marqo Fashion SigLIP [1], [2]. Product embeddings can fuse image and text representations (product description tokens), while bundle queries are image-based.

Training uses symmetric bidirectional contrastive loss:

$$\mathcal{L} = \frac{1}{2} \left[\text{CE}((\mathbf{Q}\mathbf{P}^\top)/\tau, \mathbf{y}) + \text{CE}((\mathbf{P}\mathbf{Q}^\top)/\tau, \mathbf{y}) \right] \quad (2)$$

where τ is temperature and $\mathbf{y} = [0, \dots, B-1]$ are in-batch positive indices.

D. Detection-Guided Bundle Encoding

When enabled (`params.use_bundle_boxes=true`), bundle images are processed with YOLO clothing detection [3]. Each detected region is encoded, and region embeddings are averaged per bundle:

$$\mathbf{e}_b = \text{norm} \left(\frac{1}{M_b} \sum_{m=1}^{M_b} \mathbf{e}_{b,m} \right) \quad (3)$$

If no boxes are detected, a full-image fallback is used. Detection results are cached in JSON for reuse across runs.

E. Validation Metrics

Validation follows multi-label retrieval with Hit@K and Recall@K (implemented in `src/utils/metrics.py`). Given ground-truth set G_b and top- K predictions $\hat{G}_b^K$:

$$\text{Recall@K}(b) = \frac{|G_b \cap \hat{G}_b^K|}{|\hat{G}_b^K|} \quad (4)$$

The reported metric is the mean across evaluated bundles.

IV. IMPLEMENTATION DETAILS

A. Configuration and Entrypoints

The project uses Hydra [4] with typed dataclasses (`src/config.py`). Main entrypoints are:

- `python -m src.train`: OpenCLIP training.
- `python -m src.infer`: pretrained baseline inference + local validation split.
- `python -m src.infer_openclip`: inference from trained OpenCLIP checkpoint.

Core runtime parameters include batch size, epochs, learning rate, AMP, gradient accumulation, recall cutoffs, and optional detection settings (`bbox_*`).

B. Code Architecture

Module	Responsibility
<code>src/train.py</code>	Hydra bootstrap and trainer dispatch.
<code>src/models/retrieval_openclip.py</code>	OpenCLIP training/validation loop and checkpointing.
<code>src/infer.py</code>	Baseline embedding extraction, retrieval, and submission export.
<code>src/infer_openclip.py</code>	OpenCLIP checkpoint inference for test bundles.
<code>src/detection.py</code>	YOLO detector wrapper and box extraction utilities.
<code>src/utils/retrieval.py</code>	Similarity and top-K retrieval helpers.
<code>src/utils/metrics.py</code>	Hit@K and Recall@K computation.
<code>preprocess_data.py</code>	Dataset integrity checks, split, manifests, and stats.
<code>offline_augment.py</code>	Offline augmentation and augmentation manifests.

TABLE II
PRIMARY MODULES AND RESPONSIBILITIES.

C. Training Loop Highlights

The OpenCLIP training implementation includes:

- 1) Dynamic device resolution (CPU/CUDA) and optional multi-GPU `DataParallel`.
- 2) Cosine learning-rate scheduler with 10% warmup.
- 3) Optional mixed precision via `torch.cuda.amp`.
- 4) Periodic logging, per-epoch validation, JSONL metric appends, and checkpointing (`epoch_X.pt`, `best.pt`).

D. Inference and Fallback Logic

Both inference scripts generate one row per predicted product and cap outputs to 15 per bundle. If bundle embeddings are unavailable (e.g., missing image), popular products from train links are used as fallback.

V. EXPERIMENTAL PROTOCOL

A. Local Evaluation Strategy

The repository provides two local validation pathways:

- 1) Baseline mode (`src/infer.py`) samples validation bundles with `infer.val_ratio` and reports `hit@K/recall@K`.
- 2) Training mode (`src/models/retrieval_openclip.py`) evaluates `recall@K` after each epoch, where $K = \text{params.recall_k}$.

Products are indexed once per validation pass, bundles are encoded, and top-K nearest neighbors are retrieved by cosine similarity.

B. Output Artifacts

Main artifacts generated by the training/inference workflow:

- `metrics.jsonl`: epoch-level train loss and validation recall.
- `best.pt`: best checkpoint by recall metric.
- `test_submission.csv`: final prediction file for competition upload.
- `val_metrics.json`: baseline run summary and local metrics.

C. Recommended Ablations

To measure impact of major components, the following ablations are recommended:

- Image-only products vs. image+text product embeddings.
- Bundle full-image encoding vs. detection-guided region aggregation.
- No offline augmentation vs. deterministic offline augmentation.
- Single-GPU vs. multi-GPU, with fixed seed and equal effective batch size.

VI. REPRODUCIBILITY

A. Environment

Base dependencies are listed in `requirements.txt`: `pandas`, `numpy`, `pillow`, `tqdm`, `requests`, `hydra-core`, `torch`, and `torchvision`. Additional runtime packages required by advanced features are:

- `open_clip_torch` for the OpenCLIP backend.
- `ultraliticsplus` for bundle clothing detection.

B. Determinism Controls

The code explicitly sets random seeds (`random`, `numpy`, `torch`). Offline augmentation uses stable hash-based seeds per asset and augmentation index. This enables repeatable preprocessing and consistent training behavior under fixed configuration.

C. Hydra Configuration

Configuration is split into:

- `config/config.yaml`: training and inference parameters.
- `config/files/file.yaml`: dataset/image/cache paths.

Hydra output directories store run-specific artifacts, making experiments traceable without manual path book-keeping.

D. Submission Contract

The final CSV must contain:

- `bundle_asset_id`
- `product_asset_id`

with up to 15 rows per bundle, sorted by model confidence.

VII. LIMITATIONS AND RISKS

- 1) **Single-view product supervision**: each product ID is represented by very limited imagery, which can reduce robustness to pose, occlusion, and lighting changes.
- 2) **Validation manifest default in training endpoint**: current `src/train.py` points both train and val manifests to `bundles_product_match_train.csv`; separate validation manifests should be passed for strict model selection.
- 3) **Detection dependency**: detection-guided mode depends on external YOLO checkpoints and introduces additional compute overhead.
- 4) **Submission calibration**: ranking quality matters more than simple top-1 correctness because evaluation keeps up to the first 15 predictions per bundle.

Recommended next improvements are hard-negative mining, ANN indexing (e.g., FAISS), richer text encoding strategies, and stronger reranking layers.

VIII. CONCLUSION

The Hack2026 repository already contains a complete retrieval stack: data preparation, deterministic augmentation, baseline retrieval, OpenCLIP training, optional region-aware encoding, and submission generation. The current design is practical and production-oriented, with explicit configuration, checkpointing, and metrics logging.

This paper serves as a technical specification of the implemented system and can be used as a foundation for future work on retrieval accuracy, latency optimization, and large-scale candidate reranking.

REFERENCES

- [1] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer, "Sigmoid loss for language image pre-training," *arXiv preprint arXiv:2303.15343*, 2023.
- [2] A. Radford, J. W. Kim, C. Hallacy *et al.*, "Learning transferable visual models from natural language supervision," in *Proceedings of the International Conference on Machine Learning*, 2021.
- [3] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics yolov8," GitHub repository, 2023.
- [4] O. Yadan, "Hydra: A framework for elegantly configuring complex applications," GitHub repository, 2019.

APPENDIX

APPENDIX: REPRODUCIBLE COMMANDS

A. Data Download

`bash download.sh`

B. Preprocessing

```
python preprocess_data.py \  
  --skip_download \  
  --out_dir data/preprocessed \  
  --val_ratio 0.1 \  
  --seed 42
```

C. Offline Augmentation

```
python offline_augment.py \  
  --bundles_manifest data/manifests/train_manifest.jsonl \  
  --products_manifest data/product_dataset.csv \  
  --products_images_dir data/product_images \  
  --out_dir data/offline_aug \  
  --bundles_num_augs 4 \  
  --products_num_augs 2 \  
  --img_size 224 \  
  --seed 42 \  
  --workers 8
```

D. Training (Hydra)

```
python -m src.train
```

E. Baseline Inference

```
python -m src.infer
```

F. OpenCLIP Inference

```
python -m src.infer_openclip \  
  --checkpoint outputs/retrieval_openclip/best.pt \  
  --submission-out outputs/retrieval_openclip/submission.csv
```